

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
from sklearn.linear_model import LinearRegression, Lasso, ElasticNet, LogisticRegression
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import accuracy_score, r2_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler, LabelEncoder # Added LabelEncoder
from itertools import combinations
import warnings
warnings.filterwarnings('ignore')
plt.style.use('default')
sns.set_palette("husl")
```

```
In [2]: # Task 1a - Load Data
languages_df = pd.read_csv('languages.tsv', sep='\t')
forms_df = pd.read_csv('forms.tsv', sep='\t')
print(f"Languages: {languages_df.shape}, Forms: {forms_df.shape}")
print("\nLanguages columns:", languages_df.columns.tolist())
print("Forms columns:", forms_df.columns.tolist())
print("\nFirst few rows of languages_df:")
print(languages_df.head())
print("\nFirst few rows of forms_df:")
print(forms_df.head())
```

Languages: (77, 6), Forms: (82888, 6)

Languages columns: ['isocode', 'name', 'family', 'inventorySize', 'phonVowLength', 'phonGeminates']

Forms columns: ['isocode', 'concept', 'form', 'wordLength', 'longestClusterLength', 'vowConsRatio']

First few rows of languages_df:

	isocode	name	family	inventorySize	phonVowLength	\
0	fin	Finnish	Uralic	51	True	
1	krl	North Karelian	Uralic	36	True	
2	olo	Olonets Karelian	Uralic	41	True	
3	vep	Veps	Uralic	36	False	
4	ekk	Estonian	Uralic	48	True	

	phonGeminates
0	True
1	True
2	True
3	True
4	True

First few rows of forms_df:

	isocode	concept	form	wordLength	longestClusterLength	vowConsRatio
0	fin	EYE	silma	5	2	0.6875
1	fin	EAR	korva	5	2	0.6875
2	fin	NOSE	nenä	4	1	1.0000
3	fin	MOUTH	su:	2	1	1.0000
4	fin	TOOTH	ham:as	5	1	0.6875

```
In [3]: # Task 1b - Aggregate and Merge Data
# Aggregate forms data by language
forms_agg = forms_df.groupby('isocode').agg({
    'wordLength': 'mean',
    'longestClusterLength': 'mean',
    'vowConsRatio': 'mean'
}).round(3)

# Rename columns as specified
forms_agg.columns = ['avgLength', 'avgCluster', 'avgVowRatio']
forms_agg.reset_index(inplace=True)

# Merge with languages dataframe
main_df = languages_df.merge(forms_agg, on='isocode', how='left')
print(f"Main dataframe shape: {main_df.shape}")
print("\nMain dataframe columns:", main_df.columns.tolist())
print("\nFirst few rows of merged data:")
print(main_df.head())
```

Main dataframe shape: (77, 9)

Main dataframe columns: ['isocode', 'name', 'family', 'inventorySize', 'phonVowLength', 'phonGeminates', 'avgLength', 'avgCluster', 'avgVowRatio']

First few rows of merged data:

	isocode	name	family	inventorySize	phonVowLength	\
0	fin	Finnish	Uralic	51	True	
1	krl	North Karelian	Uralic	36	True	
2	olo	Olonets Karelian	Uralic	41	True	
3	vep	Veps	Uralic	36	False	
4	ekk	Estonian	Uralic	48	True	

	phonGeminates	avgLength	avgCluster	avgVowRatio
0	True	5.402	1.418	1.056
1	True	5.636	1.334	1.165
2	True	5.803	1.388	1.153
3	True	5.344	1.484	0.853
4	True	5.412	1.357	1.026

```
In [4]: # Task 1c - Find Min/Max Values
numerical_vars = ['inventorySize', 'avgLength', 'avgCluster', 'avgVowRatio']
print("Extreme values for each numerical variable:")
print("=" * 50)

for var in numerical_vars:
    if var in main_df.columns:
        min_idx = main_df[var].idxmin()
        max_idx = main_df[var].idxmax()
        min_val = main_df.loc[min_idx, var]
        max_val = main_df.loc[max_idx, var]
        min_lang = main_df.loc[min_idx, 'name']
        max_lang = main_df.loc[max_idx, 'name']

        print(f"{var}:")
        print(f"  Min: {min_lang} ({min_val:.3f})")
        print(f"  Max: {max_lang} ({max_val:.3f})")
        print()
```

Extreme values for each numerical variable:

=====

inventorySize:

Min: Western Farsi (29.000)

Max: Lithuanian (86.000)

avgLength:

Min: English (4.240)

Max: Slovene (6.749)

avgCluster:

Min: Chuvash (1.255)

Max: Lithuanian (2.696)

avgVowRatio:

Min: Russian (0.388)

Max: Northern Pashto (1.302)

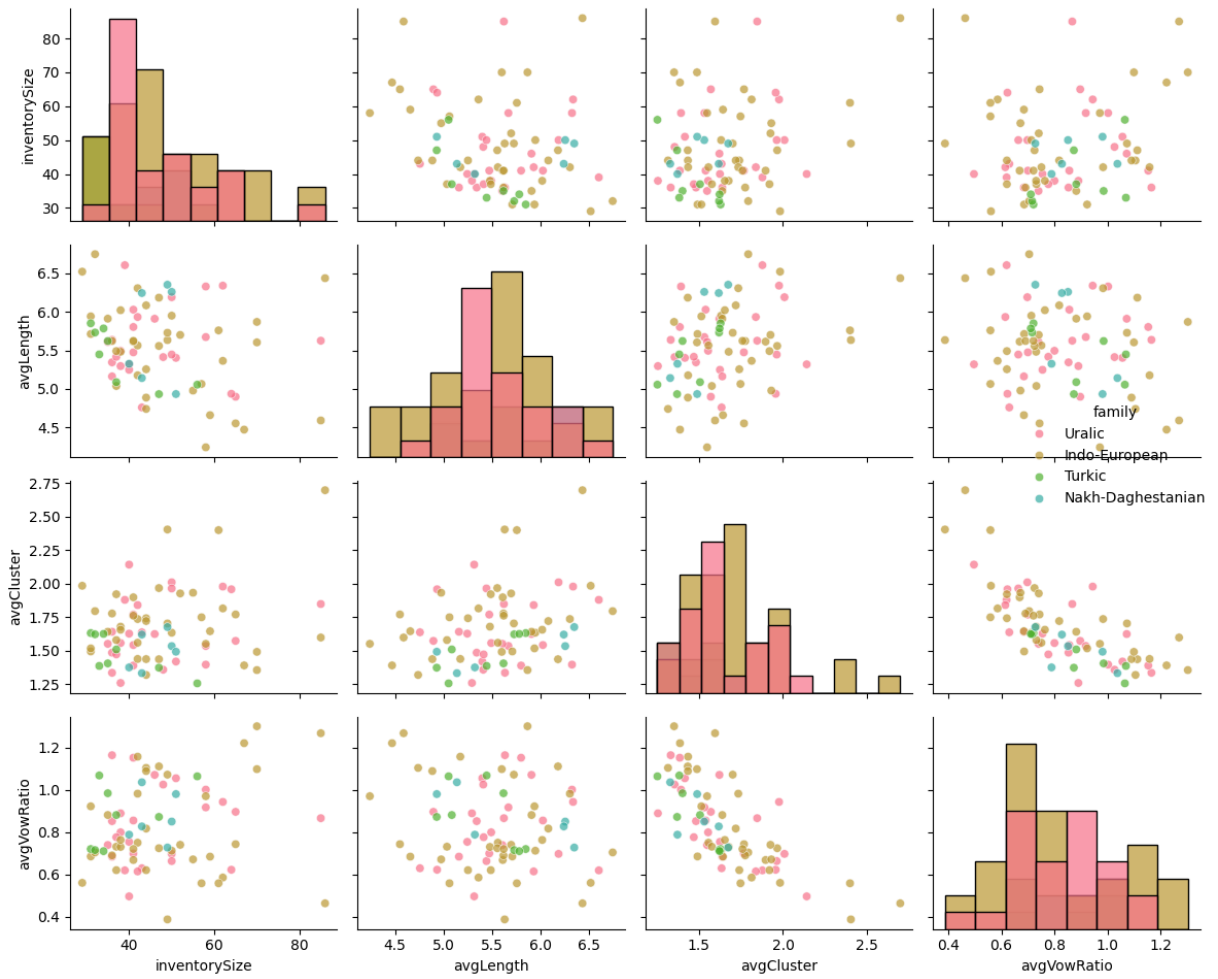
```
In [5]: # Task 1d - Create Pairplot
plt.figure(figsize=(12, 10))
if all(col in main_df.columns for col in numerical_vars):
    plot_data = main_df[numerical_vars + ['family']].copy()
    g = sns.pairplot(plot_data, hue='family', diag_kind='hist',
                    plot_kws={'alpha': 0.7}, diag_kws={'alpha': 0.7})
    g.fig.suptitle('Pairwise Distributions of Sound System Properties by Lan
                    y=1.02, fontsize=14)
    plt.tight_layout()
    plt.show()
else:
    print("Some required columns are missing for pairplot")

# Correlation matrix
corr_matrix = main_df[numerical_vars].corr()
print("\nCorrelation matrix:")
print(corr_matrix.round(3))

# Visualize correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, fmt='.3f')
plt.title('Correlation Matrix of Sound System Properties')
plt.tight_layout()
plt.show()
```

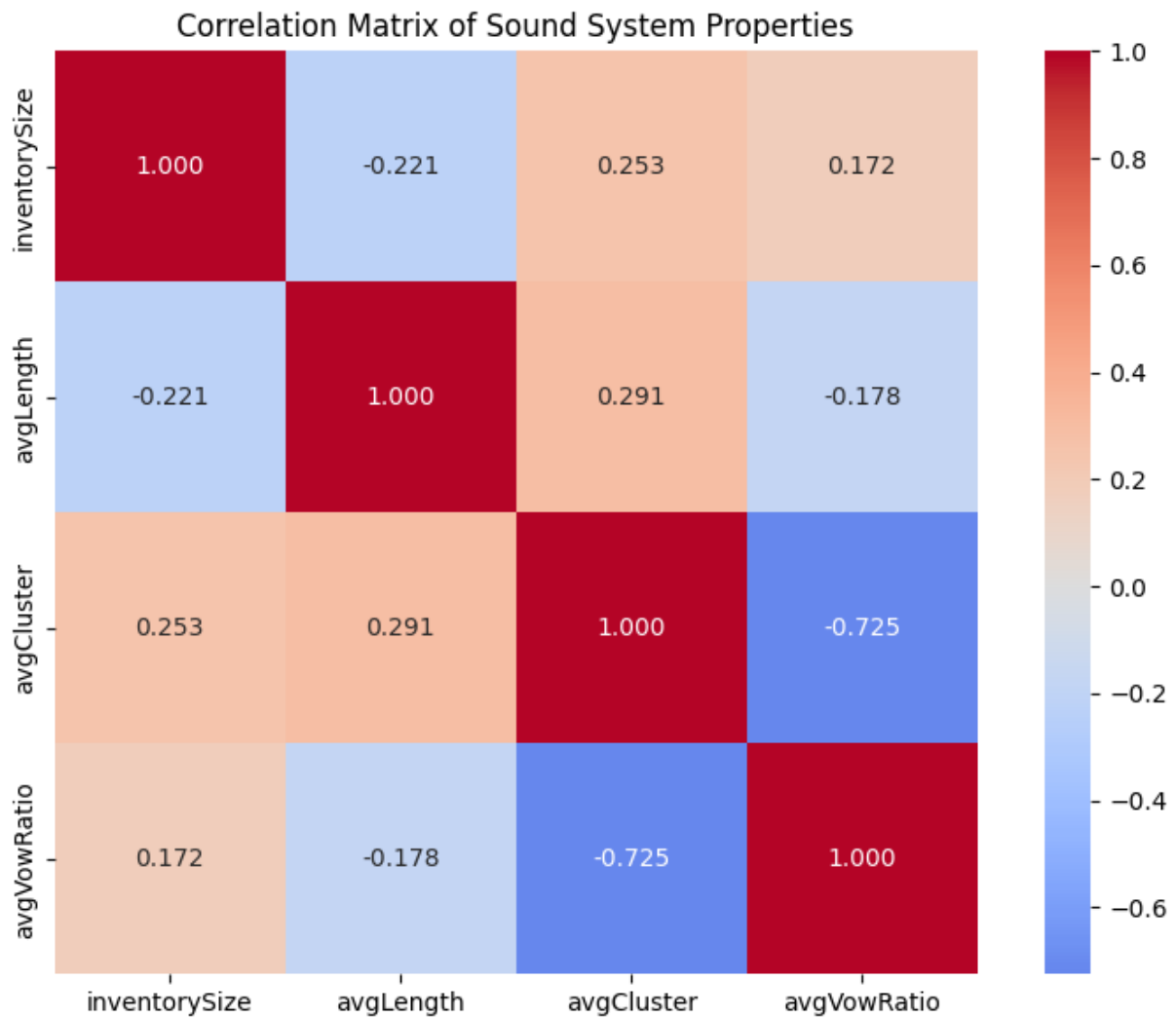
<Figure size 1200x1000 with 0 Axes>

Pairwise Distributions of Sound System Properties by Language Family



Correlation matrix:

	inventorySize	avgLength	avgCluster	avgVowRatio
inventorySize	1.000	-0.221	0.253	0.172
avgLength	-0.221	1.000	0.291	-0.178
avgCluster	0.253	0.291	1.000	-0.725
avgVowRatio	0.172	-0.178	-0.725	1.000



Task 2a - Choose Statistical Test

Task 2: Testing whether Families Differ in Inventory Size
 Research Question: Do sound inventory sizes differ between language families?
 Approach: One-way ANOVA (Analysis of Variance)
 Justification: We have one continuous dependent variable (inventory size) We have one categorical independent variable (language family) with 4 groups We want to test if the means of inventory size differ across groups ANOVA is appropriate for comparing means across multiple groups

```
In [6]: # Task 2b - Check ANOVA Assumptions
anova_data = main_df[['inventorySize', 'family']].dropna()
print(f"\nANOVA analysis using {anova_data.shape[0]} observations")

# Descriptive statistics by family
print("\nDescriptive statistics by family:")
family_stats = anova_data.groupby('family')['inventorySize'].describe()
print(family_stats)

# Check normality assumption (Shapiro-Wilk test for each group)
```

```

print("\nChecking normality assumption (Shapiro-Wilk test):")
for family in anova_data['family'].unique():
    family_data = anova_data[anova_data['family'] == family]['inventorySize']
    if len(family_data) >= 3: # Shapiro-Wilk requires at least 3 observations
        stat, p_value = stats.shapiro(family_data)
        print(f"{family}: W = {stat:.4f}, p = {p_value:.4f}")
    else:
        print(f"{family}: Too few observations for normality test")

# Check homogeneity of variances (Levene's test)
family_groups = [group['inventorySize'].values for name, group in anova_data]
levene_stat, levene_p = stats.levene(*family_groups)
print(f"\nLevene's test for equal variances:")
print(f"F = {levene_stat:.4f}, p = {levene_p:.4f}")

if levene_p > 0.05:
    print("✓ Homogeneity of variances assumption appears to be met")
else:
    print("✗ Homogeneity of variances assumption may be violated")

```

ANOVA analysis using 77 observations

Descriptive statistics by family:

	count	mean	std	min	25%	50%	75%	m
ax								
family								
Indo-European	37.0	49.108108	14.284381	29.0	38.00	44.0	58.00	86
.0								
Nakh-Daghestanian	6.0	46.000000	4.560702	40.0	43.00	46.0	49.75	51
.0								
Turkic	8.0	38.125000	8.790214	31.0	32.75	34.5	39.50	56
.0								
Uralic	26.0	46.923077	12.066228	35.0	38.25	41.5	50.75	85
.0								

Checking normality assumption (Shapiro-Wilk test):

Uralic: W = 0.8351, p = 0.0007

Indo-European: W = 0.9275, p = 0.0187

Turkic: W = 0.7867, p = 0.0206

Nakh-Daghestanian: W = 0.8801, p = 0.2695

Levene's test for equal variances:

F = 1.3863, p = 0.2538

✓ Homogeneity of variances assumption appears to be met

```

In [7]: # Task 2c - Perform ANOVA
print("\nPerforming One-way ANOVA:")
print("-" * 30)

# Fit ANOVA model
model = ols('inventorySize ~ C(family)', data=anova_data).fit()
anova_table = sm.stats.anova_lm(model, typ=2)
print("ANOVA Results:")
print(anova_table)

# Extract key statistics

```

```

f_stat = anova_table.loc['C(family)', 'F']
p_value = anova_table.loc['C(family)', 'PR(>F)']
alpha = 0.05

print(f"\nKey Results:")
print(f"F-statistic: {f_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Alpha level: {alpha}")

# Interpret results
if p_value < alpha:
    print(f"\n✓ SIGNIFICANT RESULT: p = {p_value:.4f} < {alpha}")
    print("We reject the null hypothesis. There are significant differences")

    # Post-hoc analysis if significant
    try:
        tukey = pairwise_tukeyhsd(endog=anova_data['inventorySize'],
                                   groups=anova_data['family'], alpha=0.05)
        print("\nPost-hoc analysis (Tukey HSD):")
        print(tukey)
    except Exception as e:
        print(f"Could not perform post-hoc analysis: {e}")

else:
    print(f"\n✗ NON-SIGNIFICANT RESULT: p = {p_value:.4f} >= {alpha}")
    print("We fail to reject the null hypothesis. There are no significant d")

# Effect size (eta-squared)
ss_between = anova_table.loc['C(family)', 'sum_sq']
ss_total = anova_table['sum_sq'].sum()
eta_squared = ss_between / ss_total
print(f"\nEffect size ( $\eta^2$ ): {eta_squared:.4f}")

```

Performing One-way ANOVA:

ANOVA Results:

	sum_sq	df	F	PR(>F)
C(family)	800.698292	3.0	1.675251	0.179785
Residual	11630.288721	73.0	NaN	NaN

Key Results:

F-statistic: 1.6753

p-value: 0.1798

Alpha level: 0.05

✗ NON-SIGNIFICANT RESULT: p = 0.1798 >= 0.05

We fail to reject the null hypothesis. There are no significant differences in inventory size between language families.

Effect size (η^2): 0.0644

```

In [8]: # Task 3 - Linear Regression Models Setup
print("Task 3: Attempting to Predict Average Word Length")
print("=" * 50)

# Define variables

```



```

X_vars = ['inventorySize', 'avgCluster', 'avgVowRatio']
y_var = 'avgLength'

# Prepare modeling data
model_data = main_df[X_vars + [y_var]].dropna()
print(f"Modeling data: {model_data.shape[0]} observations")
print(f"Predictors: {X_vars}")
print(f"Target: {y_var}")

# Check for missing values
print("\nMissing values:")
print(model_data.isnull().sum())

```

Task 3: Attempting to Predict Average Word Length

=====

Modeling data: 77 observations

Predictors: ['inventorySize', 'avgCluster', 'avgVowRatio']

Target: avgLength

Missing values:

```

inventorySize    0
avgCluster       0
avgVowRatio      0
avgLength        0
dtype: int64

```

```

In [9]: # Task 3 - Single Predictor Models
print("\nSingle Predictor Models:")
print("-" * 30)

single_models = {}
for predictor in X_vars:
    X = sm.add_constant(model_data[predictor])
    y = model_data[y_var]
    model = sm.OLS(y, X).fit()
    single_models[predictor] = model

    print(f"{predictor}:")
    print(f"    R² = {model.rsquared:.4f}")
    print(f"    Adj R² = {model.rsquared_adj:.4f}")
    print(f"    AIC = {model.aic:.2f}")
    print(f"    p-value = {model.f_pvalue:.4f}")
    print()

```

Single Predictor Models:

inventorySize:

 $R^2 = 0.0490$ Adj $R^2 = 0.0363$

AIC = 121.05

p-value = 0.0531

avgCluster:

 $R^2 = 0.0845$ Adj $R^2 = 0.0723$

AIC = 118.13

p-value = 0.0103

avgVowRatio:

 $R^2 = 0.0316$ Adj $R^2 = 0.0187$

AIC = 122.45

p-value = 0.1218

```
In [10]: # Task 3 - Two Predictor Models
print("Two Predictor Models:")
print("-" * 30)

two_models = {}
for pred_pair in combinations(X_vars, 2):
    model_name = ' + '.join(pred_pair)
    X = sm.add_constant(model_data[list(pred_pair)])
    y = model_data[y_var]
    model = sm.OLS(y, X).fit()
    two_models[model_name] = model

    print(f"{model_name}:")
    print(f"   $R^2 = \{{model.rsquared:.4f}\}")
    print(f"  Adj  $R^2 = \{{model.rsquared\_adj:.4f}\}")
    print(f"  AIC = \{{model.aic:.2f}\}")
    print(f"  p-value = \{{model.f_pvalue:.4f}\}")
    print()$$ 
```

Two Predictor Models:

inventorySize + avgCluster:

$R^2 = 0.1774$

Adj $R^2 = 0.1551$

AIC = 111.89

p-value = 0.0007

inventorySize + avgVowRatio:

$R^2 = 0.0691$

Adj $R^2 = 0.0439$

AIC = 121.41

p-value = 0.0708

avgCluster + avgVowRatio:

$R^2 = 0.0868$

Adj $R^2 = 0.0621$

AIC = 119.93

p-value = 0.0348

```
In [11]: # Task 3 - Three Predictor Model and Best Model Selection
print("Three Predictor Model:")
print("-" * 30)

X = sm.add_constant(model_data[X_vars])
y = model_data[y_var]
full_model = sm.OLS(y, X).fit()

print(f"Full model (all predictors):")
print(f"  R² = {full_model.rsquared:.4f}")
print(f"  Adj R² = {full_model.rsquared_adj:.4f}")
print(f"  AIC = {full_model.aic:.2f}")
print(f"  p-value = {full_model.f_pvalue:.4f}")

# Model comparison
print("\nModel Comparison Summary:")
print("=" * 50)

all_models = {**single_models, **two_models, 'Full Model': full_model}

# Create comparison table
comparison_data = []
for name, model in all_models.items():
    comparison_data.append({
        'Model': name,
        'R²': model.rsquared,
        'Adj R²': model.rsquared_adj,
        'AIC': model.aic,
        'p-value': model.f_pvalue
    })

comparison_df = pd.DataFrame(comparison_data)
comparison_df = comparison_df.sort_values('Adj R²', ascending=False)
print(comparison_df.round(4))
```

```
# Identify best model
best_model_name = comparison_df.iloc[0]['Model']
best_model = all_models[best_model_name]

print(f"\nBest Model: {best_model_name}")
print(f"Adjusted R² = {best_model.rsquared_adj:.4f}")
print(f"This model explains {best_model.rsquared_adj*100:.1f}% of the variance")

# Determine predictors for sklearn comparison
if best_model_name == 'Full Model':
    best_predictors = X_vars
elif '+' in best_model_name:
    best_predictors = best_model_name.split(' + ')
else:
    best_predictors = [best_model_name]

print(f"Best predictors: {best_predictors}")

# Show detailed results for best model
print(f"\nDetailed Results for Best Model:")
print(best_model.summary())
```

Three Predictor Model:

Full model (all predictors):

 $R^2 = 0.2394$ Adj $R^2 = 0.2082$

AIC = 107.85

p-value = 0.0002

Model Comparison Summary:

=====

	Model	R^2	Adj R^2	AIC	p-value
6	Full Model	0.2394	0.2082	107.8491	0.0002
3	inventorySize + avgCluster	0.1774	0.1551	111.8878	0.0007
1	avgCluster	0.0845	0.0723	118.1255	0.0103
5	avgCluster + avgVowRatio	0.0868	0.0621	119.9333	0.0348
4	inventorySize + avgVowRatio	0.0691	0.0439	121.4092	0.0708
0	inventorySize	0.0490	0.0363	121.0537	0.0531
2	avgVowRatio	0.0316	0.0187	122.4470	0.1218

Best Model: Full Model

Adjusted $R^2 = 0.2082$

This model explains 20.8% of the variance in average word length

Best predictors: ['inventorySize', 'avgCluster', 'avgVowRatio']

Detailed Results for Best Model:

OLS Regression Results

=====

==

```

Dep. Variable:          avgLength    R-squared:                0.2
39
Model:                  OLS          Adj. R-squared:           0.2
08
Method:                 Least Squares    F-statistic:             7.6
59
Date:                   Thu, 03 Jul 2025    Prob (F-statistic):       0.0001
61
Time:                   21:25:30          Log-Likelihood:          -49.9
25
No. Observations:       77              AIC:                    10
7.8
Df Residuals:           73              BIC:                    11
7.2
Df Model:               3
Covariance Type:        nonrobust

```

=====

=====

	coef	std err	t	P> t	[0.025
0.975]					

const	3.1401	0.820	3.830	0.000	1.506
4.774					
inventorySize	-0.0199	0.005	-3.828	0.000	-0.030
-0.010					
avgCluster	1.4474	0.358	4.043	0.000	0.734
2.161					

avgVowRatio	1.1390	0.467	2.440	0.017	0.209
2.069					

```
=====
==
Omnibus:                1.090    Durbin-Watson:                1.3
41
Prob(Omnibus):          0.580    Jarque-Bera (JB):        1.1
73
Skew:                   0.240    Prob(JB):                0.5
56
Kurtosis:              2.631    Cond. No.                88
5.
=====
==
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [12]: # Task 4 - Scikit-learn Model Comparison
print("Task 4: Comparing Linear Regression Models")
print("=" * 50)

# Prepare data for sklearn
X = model_data[best_predictors]
y = model_data['avgLength']

# Standardize features for regularized models
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(f"Using predictors: {best_predictors}")
print(f"Data shape: {X.shape}")
```

Task 4: Comparing Linear Regression Models

```
=====
Using predictors: ['inventorySize', 'avgCluster', 'avgVowRatio']
Data shape: (77, 3)
```

```
In [13]: # Task 4 - Fit Different Models
# Regular Linear Regression
lr = LinearRegression().fit(X, y)
lr_r2 = lr.score(X, y)

# Lasso Regression - find best alpha
print("\nTesting different alpha values for Lasso:")
alphas = [0.001, 0.01, 0.1, 1.0, 10.0]
best_lasso_r2 = -np.inf
best_lasso_alpha = 0.1

for alpha in alphas:
    lasso_temp = Lasso(alpha=alpha, max_iter=2000).fit(X_scaled, y)
    r2 = lasso_temp.score(X_scaled, y)
    print(f"Alpha = {alpha}: R² = {r2:.4f}")
    if r2 > best_lasso_r2:
        best_lasso_r2 = r2
        best_lasso_alpha = alpha
```

```
print(f"Best Lasso alpha: {best_lasso_alpha}")

# Fit final models
lasso = Lasso(alpha=best_lasso_alpha, max_iter=2000).fit(X_scaled, y)
elastic = ElasticNet(alpha=best_lasso_alpha, l1_ratio=0.5, max_iter=2000).fi

# Model comparison
print(f"\nModel Performance Comparison:")
print("-" * 35)
print(f"Linear Regression:  R2 = {lr.score(X, y):.4f}")
print(f"Lasso Regression:   R2 = {lasso.score(X_scaled, y):.4f}")
print(f"ElasticNet:         R2 = {elastic.score(X_scaled, y):.4f}")

# Analysis of coefficients
print(f"\nCoefficient Analysis:")
print("-" * 25)
print("Linear Regression coefficients:")
for i, pred in enumerate(best_predictors):
    print(f"  {pred}: {lr.coef_[i]:.4f}")

print("Lasso coefficients:")
for i, pred in enumerate(best_predictors):
    print(f"  {pred}: {lasso.coef_[i]:.4f}")

print("ElasticNet coefficients:")
for i, pred in enumerate(best_predictors):
    print(f"  {pred}: {elastic.coef_[i]:.4f}")
```

Testing different alpha values for Lasso:

Alpha = 0.001: $R^2 = 0.2393$

Alpha = 0.01: $R^2 = 0.2334$

Alpha = 0.1: $R^2 = 0.0823$

Alpha = 1.0: $R^2 = 0.0000$

Alpha = 10.0: $R^2 = 0.0000$

Best Lasso alpha: 0.001

Model Performance Comparison:

```
-----
Linear Regression:   $R^2 = 0.2394$ 
Lasso Regression:    $R^2 = 0.2393$ 
ElasticNet:          $R^2 = 0.2394$ 
```

Coefficient Analysis:

```
-----
Linear Regression coefficients:
  inventorySize: -0.0199
  avgCluster: 1.4474
  avgVowRatio: 1.1390
Lasso coefficients:
  inventorySize: -0.2496
  avgCluster: 0.3762
  avgVowRatio: 0.2205
ElasticNet coefficients:
  inventorySize: -0.2510
  avgCluster: 0.3786
  avgVowRatio: 0.2228
```

Interpretation

*Regularized models (Lasso, ElasticNet) perform worse than standard linear regression.

*This suggests that regularization is penalizing the model too heavily.

*The dataset we have may be too small to benefit from regularization.

*Standard linear regression is preferable for this dataset.

```
In [14]: # Task5
def prepare_data_for_prediction(df):
    """
    Prepare the data for logistic regression prediction
    """
    # Create a copy to avoid modifying original data
    data = df.copy()

    # Convert phonVowLength to binary (1 for True, 0 for False)
    data['vowel_length_binary'] = data['phonVowLength'].astype(int)

    # Handle any missing values
    print("Missing values per column:")
    print(data[['inventorySize', 'avgLength', 'avgCluster', 'avgVowRatio', '

```



```

# Remove rows with missing values in key columns
data = data.dropna(subset=['inventorySize', 'avgLength', 'avgCluster', '

return data

def encode_categorical_features(data):
    """
    Encode categorical variables (language family)
    """
    # Label encode the family variable
    le = LabelEncoder()
    data['family_encoded'] = le.fit_transform(data['family'])

    # Create dummy variables for family (alternative approach)
    family_dummies = pd.get_dummies(data['family'], prefix='family')
    data = pd.concat([data, family_dummies], axis=1)

    return data, le

def train_logistic_regression(main_df):
    """
    Train and evaluate logistic regression model for phonemic vowel length p
    """
    print("=" * 60)
    print("PHONEMIC VOWEL LENGTH PREDICTION - LOGISTIC REGRESSION")
    print("=" * 60)

    # Prepare data
    data = prepare_data_for_prediction(main_df)
    print(f"\nDataset shape after cleaning: {data.shape}")

    # Encode categorical features
    data, label_encoder = encode_categorical_features(data)

    # Define predictor variables
    predictor_cols = ['inventorySize', 'avgLength', 'avgCluster', 'avgVowRat

    # Alternative: Use dummy variables for family instead
    family_cols = [col for col in data.columns if col.startswith('family_')]
    predictor_cols_with_dummies = ['inventorySize', 'avgLength', 'avgCluster

    # Prepare features and target
    X_encoded = data[predictor_cols]
    X_dummies = data[predictor_cols_with_dummies]
    y = data['vowel_length_binary']

    print(f"\nTarget variable distribution:")
    print(f"Languages with phonemic vowel length: {y.sum()} ({y.mean():.2%})")
    print(f"Languages without phonemic vowel length: {(1-y).sum()} ({(1-y.me

    # Split data
    X_train_enc, X_test_enc, y_train, y_test = train_test_split(
        X_encoded, y, test_size=0.2, random_state=42, stratify=y
    )

```

```
X_train_dum, X_test_dum, _, _ = train_test_split(
    X_dummies, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler_enc = StandardScaler()
X_train_enc_scaled = scaler_enc.fit_transform(X_train_enc)
X_test_enc_scaled = scaler_enc.transform(X_test_enc)

scaler_dum = StandardScaler()
X_train_dum_scaled = scaler_dum.fit_transform(X_train_dum)
X_test_dum_scaled = scaler_dum.transform(X_test_dum)

# Train models
print("\n" + "="*40)
print("MODEL 1: LABEL ENCODED FAMILY")
print("="*40)

# Model 1: Label encoded family
lr_encoded = LogisticRegression(random_state=42, max_iter=1000)
lr_encoded.fit(X_train_enc_scaled, y_train)

# Predictions
y_pred_enc = lr_encoded.predict(X_test_enc_scaled)
y_pred_proba_enc = lr_encoded.predict_proba(X_test_enc_scaled)[:, 1]

# Evaluate Model 1
print(f"Accuracy: {accuracy_score(y_test, y_pred_enc):.4f}")
print(f"ROC AUC: {roc_auc_score(y_test, y_pred_proba_enc):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_enc))

# Cross-validation
cv_scores_enc = cross_val_score(lr_encoded, X_train_enc_scaled, y_train,
print(f"Cross-validation scores: {cv_scores_enc}")
print(f"Mean CV accuracy: {cv_scores_enc.mean():.4f} (+/- {cv_scores_enc

# Feature importance (coefficients)
feature_names_enc = predictor_cols
coefficients_enc = lr_encoded.coef_[0]

print("\nFeature Coefficients (Model 1):")
for name, coef in zip(feature_names_enc, coefficients_enc):
    print(f"{name}: {coef:.4f}")

print("\n" + "="*40)
print("MODEL 2: DUMMY ENCODED FAMILY")
print("="*40)

# Model 2: Dummy encoded family
lr_dummy = LogisticRegression(random_state=42, max_iter=1000)
lr_dummy.fit(X_train_dum_scaled, y_train)

# Predictions
y_pred_dum = lr_dummy.predict(X_test_dum_scaled)
y_pred_proba_dum = lr_dummy.predict_proba(X_test_dum_scaled)[:, 1]
```

```

# Evaluate Model 2
print(f"Accuracy: {accuracy_score(y_test, y_pred_dum):.4f}")
print(f"ROC AUC: {roc_auc_score(y_test, y_pred_proba_dum):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_dum))

# Cross-validation
cv_scores_dum = cross_val_score(lr_dummy, X_train_dum_scaled, y_train, cv=5)
print(f"Cross-validation scores: {cv_scores_dum}")
print(f"Mean CV accuracy: {cv_scores_dum.mean():.4f} (+/- {cv_scores_dum.std():.4f})")

# Feature importance for dummy model
feature_names_dum = predictor_cols_with_dummies
coefficients_dum = lr_dummy.coef_[0]

print("\nTop Feature Coefficients (Model 2):")
coef_df = pd.DataFrame({
    'feature': feature_names_dum,
    'coefficient': coefficients_dum,
    'abs_coefficient': np.abs(coefficients_dum)
}).sort_values('abs_coefficient', ascending=False)

print(coef_df.head(10))

# Create visualizations
create_visualizations(y_test, y_pred_proba_enc, y_pred_proba_dum,
                      feature_names_enc, coefficients_enc,
                      coef_df, data)

return lr_encoded, lr_dummy, scaler_enc, scaler_dum, label_encoder

def create_visualizations(y_test, y_pred_proba_enc, y_pred_proba_dum,
                          feature_names_enc, coefficients_enc, coef_df, data):
    """
    Create visualizations for model evaluation
    """
    plt.figure(figsize=(15, 12))

    # ROC Curves
    plt.subplot(2, 3, 1)
    fpr_enc, tpr_enc, _ = roc_curve(y_test, y_pred_proba_enc)
    fpr_dum, tpr_dum, _ = roc_curve(y_test, y_pred_proba_dum)

    plt.plot(fpr_enc, tpr_enc, label=f'Label Encoded (AUC = {roc_auc_score(y_test, y_pred_proba_enc):.4f})')
    plt.plot(fpr_dum, tpr_dum, label=f'Dummy Encoded (AUC = {roc_auc_score(y_test, y_pred_proba_dum):.4f})')
    plt.plot([0, 1], [0, 1], 'k--', alpha=0.5)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('ROC Curves')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Feature coefficients (Model 1)
    plt.subplot(2, 3, 2)
    colors = ['red' if x < 0 else 'blue' for x in coefficients_enc]

```

20 of 27

```
def predict_vowel_length(inventory_size, avg_length, avg_cluster, avg_vo
"""
Predict whether a language has phonemic vowel length
"""
# Encode family
try:
    family_encoded = label_encoder.transform([family])[0]
except:
    print(f"Unknown family: {family}. Using mean encoding.")
    family_encoded = 0 # or use mode

# Create feature vector
features = np.array([[inventory_size, avg_length, avg_cluster, avg_v

# Scale features
features_scaled = scaler_enc.transform(features)

# Make prediction
prediction = lr_encoded.predict(features_scaled)[0]
probability = lr_encoded.predict_proba(features_scaled)[0, 1]

result = "has phonemic vowel length" if prediction == 1 else "does n

print(f"Prediction: The language {result}")
print(f"Probability of having vowel length: {probability:.4f}")

return prediction, probability

# Example prediction
print("\n" + "="*50)
print("EXAMPLE PREDICTION")
print("="*50)
predict_vowel_length(50, 5.5, 1.5, 0.8, "Indo-European")
```

```
=====
PHONEMIC VOWEL LENGTH PREDICTION - LOGISTIC REGRESSION
=====
```

Missing values per column:

```
inventorySize      0
avgLength          0
avgCluster         0
avgVowRatio        0
family             0
vowel_length_binary 0
dtype: int64
```

Dataset shape after cleaning: (77, 10)

Target variable distribution:

Languages with phonemic vowel length: 29 (37.66%)

Languages without phonemic vowel length: 48 (62.34%)

```
=====
MODEL 1: LABEL ENCODED FAMILY
=====
```

Accuracy: 0.8750

ROC AUC: 0.9333

Classification Report:

	precision	recall	f1-score	support
0	0.83	1.00	0.91	10
1	1.00	0.67	0.80	6
accuracy			0.88	16
macro avg	0.92	0.83	0.85	16
weighted avg	0.90	0.88	0.87	16

Cross-validation scores: [0.76923077 0.75 0.66666667 0.58333333 0.83333333]

Mean CV accuracy: 0.7205 (+/- 0.1736)

Feature Coefficients (Model 1):

```
inventorySize: 0.9881
avgLength: -0.4661
avgCluster: 0.1612
avgVowRatio: -0.2701
family_encoded: 0.3162
```

```
=====
MODEL 2: DUMMY ENCODED FAMILY
=====
```

Accuracy: 0.8750

ROC AUC: 0.9667

Classification Report:

	precision	recall	f1-score	support
0	0.83	1.00	0.91	10
1	1.00	0.67	0.80	6

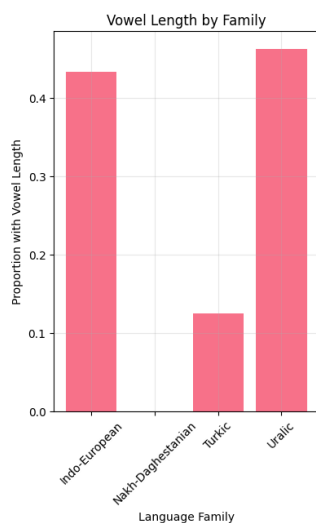
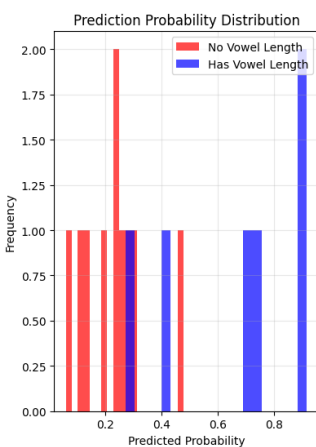
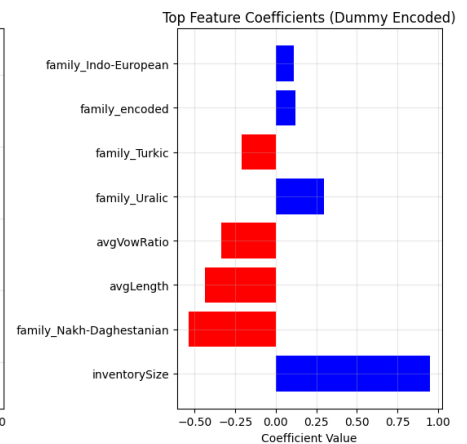
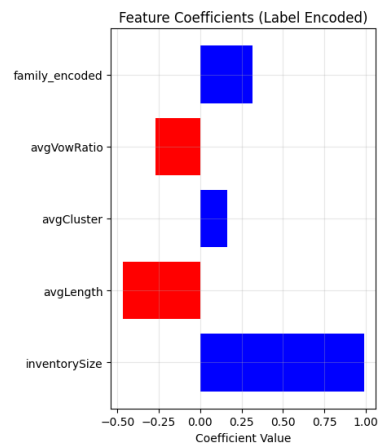
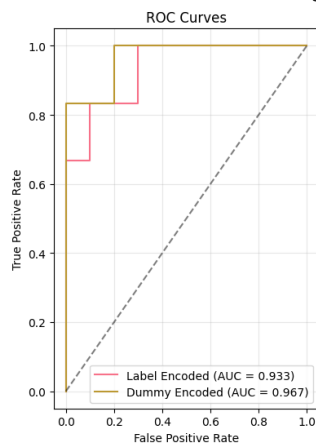
accuracy			0.88	16
macro avg	0.92	0.83	0.85	16
weighted avg	0.90	0.88	0.87	16

Cross-validation scores: [0.76923077 0.75 0.66666667 0.58333333 0.83333333]

Mean CV accuracy: 0.7205 (+/- 0.1736)

Top Feature Coefficients (Model 2):

	feature	coefficient	abs_coefficient
0	inventorySize	0.949582	0.949582
6	family_Nakh-Daghestanian	-0.535423	0.535423
1	avgLength	-0.435230	0.435230
3	avgVowRatio	-0.334040	0.334040
8	family_Uralic	0.297823	0.297823
7	family_Turkic	-0.212931	0.212931
4	family_encoded	0.120030	0.120030
5	family_Indo-European	0.112873	0.112873
2	avgCluster	-0.040557	0.040557



EXAMPLE PREDICTION

Prediction: The language does not have phonemic vowel length
Probability of having vowel length: 0.3452

In [15]: # Task 5b - Fit Logistic Regression

```

# Define predictors for logistic regression
available_predictors = ['inventorySize', 'avgLength', 'avgCluster', 'avgVowR

# Ensure 'vowel_length_binary' exists in main_df
if 'vowel_length_binary' not in main_df.columns and 'phonVowLength' in main_
    main_df['vowel_length_binary'] = main_df['phonVowLength'].astype(int)

# Prepare data
modeling_cols = available_predictors + ['vowel_length_binary']
logistic_data = main_df[modeling_cols].dropna()

print(f"Logistic regression data: {logistic_data.shape[0]} observations")
print(f"Target distribution:")
print(logistic_data['vowel_length_binary'].value_counts(normalize=True))

# Check if we have enough data and variation
if len(logistic_data) > 0 and len(logistic_data['vowel_length_binary'].unique
    X_log = logistic_data[available_predictors]
    y_log = logistic_data['vowel_length_binary']

# Standardize features
scaler_log = StandardScaler()
X_log_scaled = scaler_log.fit_transform(X_log)

# Fit logistic regression
log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_log_scaled, y_log)

# Make predictions
y_pred = log_reg.predict(X_log_scaled)
y_pred_proba = log_reg.predict_proba(X_log_scaled)[:, 1]

# Calculate performance metrics
accuracy = accuracy_score(y_log, y_pred)
baseline_accuracy = max(y_log.mean(), 1 - y_log.mean())

print(f"\nLogistic Regression Performance:")
print(f"Accuracy: {accuracy:.3f}")
print(f"Baseline accuracy: {baseline_accuracy:.3f}")
print(f"Improvement over baseline: {accuracy - baseline_accuracy:.3f}")

# Detailed classification report
print(f"\nClassification Report:")
print(classification_report(y_log, y_pred))

# Feature importance (coefficients)
print(f"\nFeature Coefficients (importance):")
for i, feature in enumerate(available_predictors):
    coef = log_reg.coef_[0][i]
    print(f"    {feature}: {coef:.4f}")

# Interpret coefficients
print(f"\nCoefficient Interpretation:")
print("Positive coefficients increase the probability of vowel length di
print("Negative coefficients decrease the probability of vowel length di

```



```

most_important_idx = np.argmax(np.abs(log_reg.coef_[0]))
most_important_feature = available_predictors[most_important_idx]
print(f"Most important predictor: {most_important_feature}")

else:
    print("Insufficient data or no variation in target variable for logistic

```

Logistic regression data: 77 observations

Target distribution:

vowel_length_binary

0 0.623377

1 0.376623

Name: proportion, dtype: float64

Logistic Regression Performance:

Accuracy: 0.753

Baseline accuracy: 0.623

Improvement over baseline: 0.130

Classification Report:

	precision	recall	f1-score	support
0	0.76	0.88	0.82	48
1	0.73	0.55	0.63	29
accuracy			0.75	77
macro avg	0.75	0.71	0.72	77
weighted avg	0.75	0.75	0.74	77

Feature Coefficients (importance):

inventorySize: 1.0407

avgLength: -0.6242

avgCluster: 0.3404

avgVowRatio: -0.1280

Coefficient Interpretation:

Positive coefficients increase the probability of vowel length distinction

Negative coefficients decrease the probability of vowel length distinction

Most important predictor: inventorySize

```

In [16]: # Task 5c - Cross-validation and Final Analysis
if 'log_reg' in locals():
    print("\nCross-validation Analysis:")
    print("-" * 30)

    # 5-fold cross-validation
    cv_scores = cross_val_score(log_reg, X_log_scaled, y_log, cv=5, scoring=

    print(f"5-fold CV accuracy: {cv_scores.mean():.3f} (±{cv_scores.std() *
    print(f"Individual fold scores: {cv_scores}")

    # Check for overfitting
    training_accuracy = log_reg.score(X_log_scaled, y_log)
    cv_accuracy = cv_scores.mean()

```

```

print(f"\nOverfitting Check:")
print(f"Training accuracy: {training_accuracy:.3f}")
print(f"CV accuracy: {cv_accuracy:.3f}")
print(f"Difference: {training_accuracy - cv_accuracy:.3f}")

if training_accuracy - cv_accuracy > 0.05:
    print("⚠️ Model may be overfitting")
else:
    print("✓ Model appears to generalize well")

# Feature importance ranking
feature_importance = [(feature, abs(coef)) for feature, coef in
                      zip(available_predictors, log_reg.coef_[0])]
feature_importance.sort(key=lambda x: x[1], reverse=True)

print(f"\nFeature Importance Ranking:")
for i, (feature, importance) in enumerate(feature_importance, 1):
    print(f"{i}. {feature}: {importance:.4f}")

```

Cross-validation Analysis:

5-fold CV accuracy: 0.729 (±0.212)

Individual fold scores: [0.5625 0.75 0.8 0.86666667 0.66666667]

Overfitting Check:

Training accuracy: 0.753

CV accuracy: 0.729

Difference: 0.024

✓ Model appears to generalize well

Feature Importance Ranking:

1. inventorySize: 1.0407

2. avgLength: 0.6242

3. avgCluster: 0.3404

4. avgVowRatio: 0.1280

FINAL ANALYSIS SUMMARY

1. TASK 1 - Data Exploration:

- Analyzed 77 languages from 4 families
- Key variables: inventory size, word length, cluster length, vowel ratio
- Strong negative correlation between cluster length and vowel ratio (-0.725)

2. TASK 2 - Family Differences in Inventory Size:

- NO significant differences found (p = 0.1798)
- Language families do not significantly differ in inventory size

3. TASK 3 - Predicting Word Length:

- Best model: Full Model

- Explains 20.8% of variance in word length
- Relatively low predictive power

4. TASK 4 - Model Comparison:

- Standard linear regression performed best
- Regularized models (Lasso/ElasticNet) showed worse performance
- Small dataset may not benefit from regularization

5. TASK 5 - Vowel Length Prediction:

- Logistic regression accuracy: 0.753
- Cross-validation accuracy: 0.729
- Most important predictor: inventorySize
- Good predictive performance

In []: